

```

# =====
# BLOB Principe - SAUT 3.7 : THERMODYNAMIQUE NON-EQUILIBRE
# =====
#
# Objectif : tester l hypothese que le Blob est un systeme HORS-EQUILIBRE
# stationnaire conservatif. Cette hypothese a emergee de Saut 3.5v3 ou
# Q3 Blob / Q3 Null = 0.31 a N=500 (le Blob dissipe MOINS que le hasard).
#
# Trois batteries de tests :
#
# T1 - PRODUCTION D ENTROPIE LOCALE (Schnakenberg / Seifert)
#   Mesure du taux de production d entropie  $\sigma_t = \langle Q_{in} - Q_{out} \rangle / \langle pas \rangle$ 
#   Si  $\sigma_t > 0$  stable -> systeme hors equilibre stationnaire (NESS)
#
# T2 - FLUCTUATION-DISSIPATION GENERALISEE
#   Test du theoreme de Crooks :  $P(\Delta S_{traj}) / P(-\Delta S_{traj}) = \exp(\Delta S)$ 
#   Si la relation est respectee : equilibre. Si elle est violee : hors equilibre
#
# T3 - REPONSE LINEAIRE / ONSAGER
#   Perturbation  $\tau \rightarrow \tau + \epsilon$ , mesure de la reponse  $\langle \Delta D \rangle$ 
#   Comparaison avec le theoreme d Onsager : symetrie des coefficients croises
#
# Configuration :
#   N=200 fixe (regime intermediaire),  $\tau=0.50$  (regime standard),
#   3 seeds, 200k pas equilibration + 500k pas mesure
#   Comparaison Blob vs Null systematique
#
# Sorties :
#   -  $\sigma_t$  (production d entropie) Blob vs Null
#   - Distribution  $P(\Delta S_{traj})$  et test Crooks
#   - Coefficient de reponse lineaire
#   - Verdict : NESS confirme / refute
# =====

```

```
import sys
```

```
import os
```

```
def auto_check():
```

```
    print("=" * 70)
```

```
    print("  AUTO-TEST CORRUPTION CARACTERES")
```

```
    print("=" * 70)
```

```
    try:
```

```
        if hasattr(sys, 'argv') and len(sys.argv) > 0 and os.path.exists(sys.argv
```

```
            source_file = sys.argv[0]
```

```

else:
    source_file = __file__ if '__file__' in globals() else None
    if source_file and os.path.exists(source_file):
        with open(source_file, 'r', encoding='utf-8') as f:
            content = f.read()
            dangerous = {chr(8216): "U+2018", chr(8217): "U+2019",
                          chr(8220): "U+201C", chr(8221): "U+201D",
                          chr(8211): "U+2013", chr(8212): "U+2014"}
            found = [(name, content.count(c)) for c, name in dangerous.items() if
                      if found:
                        print("ERREUR caracteres corrompus :", found)
                        return False
            print("OK : pas de corruption")
    else:
        print("Note : auto-test partiel")
except Exception as e:
    print("Note : " + str(e))
print("=" * 70)
return True

if not auto_check():
    sys.exit(1)

```

```

import numpy as np
import time
import json
import gc
from collections import Counter, deque

```

```

KAPPA_0 = 8.0
BETA_VIAB = 0.30
EPS_DECAL = 0.05

```

```

OUTPUT_DIR = '/content/blob_saut37_results'
os.makedirs(OUTPUT_DIR, exist_ok=True)

```

```

class BlobUniverseFixedN:

```

```

    """N fixe, mode 'blob' ou 'null'. Trace les variations de S pour mesures NESS

```

```

    def __init__(self, N, tau, seed=0, mode='blob'):
        self.rng = np.random.default_rng(seed)
        self.N = N
        self.tau = tau
        self.mode = mode
        self.seed = seed

```

```

self.positions = [self.rng.random(3) for _ in range(N)]
self.adj = [set() for _ in range(N)]
self.deg = np.zeros(N, dtype=np.int32)
self.edge_list = []
self.n_tri = 0
self.n_tetra = 0
self.S_curr = 0.0
# Tracking pour T1, T2, T3
self.delta_S_history = [] # liste des Delta S des pas acceptes
self.q_in = 0.0 # entropie absorbee (Delta S > 0)
self.q_out = 0.0 # entropie cedee (|Delta S| pour Delta S < 0)
self._initialize_edges()

def _initialize_edges(self):
    E_target = int(KAPPA_0 * self.N / 2)
    attempts = 0
    max_attempts = E_target * 100
    while len(self.edge_list) < E_target and attempts < max_attempts:
        u = int(self.rng.integers(self.N))
        v = int(self.rng.integers(self.N))
        attempts += 1
        if u == v or v in self.adj[u]:
            continue
        u, v = min(u, v), max(u, v)
        dt_add = len(self.adj[u] & self.adj[v])
        self.adj[u].add(v)
        self.adj[v].add(u)
        self.deg[u] += 1
        self.deg[v] += 1
        self.edge_list.append((u, v))
        self.n_tri += dt_add
        common = self.adj[u] & self.adj[v]
        if len(common) >= 2:
            cl = list(common)
            for i, w in enumerate(cl):
                for x in cl[i+1:]:
                    if x in self.adj[w]:
                        self.n_tetra += 1
    self.S_curr = self._compute_S()

def _compute_D(self):
    mu = float(self.deg.mean())
    if mu == 0:
        return 0.0
    return float(self.deg.std()) / mu

def _compute_S(self):

```

```

        return np.log(1 + self.n_tri) - BETA_VIAB * self._compute_D()

def _count_tetra_through_edge(self, u, v):
    common = self.adj[u] & self.adj[v]
    if len(common) < 2:
        return 0
    cl = list(common)
    count = 0
    for i, w in enumerate(cl):
        adj_w = self.adj[w]
        for x in cl[i+1:]:
            if x in adj_w:
                count += 1
    return count

def _list_tetrahedra(self):
    result = []
    seen = set()
    for u, v in self.edge_list:
        common = self.adj[u] & self.adj[v]
        if len(common) < 2:
            continue
        cl = sorted(common)
        for i, w in enumerate(cl):
            for x in cl[i+1:]:
                if x in self.adj[w]:
                    key = tuple(sorted([u, v, w, x]))
                    if key not in seen:
                        seen.add(key)
                        result.append(key)
    return result

def compute_PHI(self):
    if self.n_tetra == 0:
        return 0.0
    tetras = self._list_tetrahedra()
    if not tetras:
        return 0.0
    edge_to_tetras = {}
    for tet in tetras:
        for i in range(4):
            for j in range(i+1, 4):
                e = (tet[i], tet[j])
                edge_to_tetras.setdefault(e, []).append(tet)
    neighbors_count = {tet: set() for tet in tetras}
    for e, tlist in edge_to_tetras.items():
        if len(tlist) > 1:

```

```

        for i, t1 in enumerate(tlist):
            for t2 in tlist[i+1:]:
                neighbors_count[t1].add(t2)
                neighbors_count[t2].add(t1)
    total = sum(len(v) for v in neighbors_count.values())
    return float(total) / len(tetras)

def step_swap(self, record_delta_S=False):
    if not self.edge_list:
        return False, 0.0
    idx = int(self.rng.integers(len(self.edge_list)))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False, 0.0
    u_add, v_add = min(u_add, v_add), max(u_add, v_add)
    if (u_add, v_add) == (u_rm, v_rm):
        return False, 0.0
    dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
    dtetra_rm = self._count_tetra_through_edge(u_rm, v_rm)
    self.adj[u_rm].discard(v_rm)
    self.adj[v_rm].discard(u_rm)
    self.deg[u_rm] -= 1
    self.deg[v_rm] -= 1
    dt_add = len(self.adj[u_add] & self.adj[v_add])
    self.adj[u_add].add(v_add)
    self.adj[v_add].add(u_add)
    self.deg[u_add] += 1
    self.deg[v_add] += 1
    dtetra_add = self._count_tetra_through_edge(u_add, v_add)
    new_n_tri = self.n_tri - dt_rm + dt_add
    if new_n_tri < 0:
        # rollback
        self.adj[u_add].discard(v_add)
        self.adj[v_add].discard(u_add)
        self.deg[u_add] -= 1
        self.deg[v_add] -= 1
        self.adj[u_rm].add(v_rm)
        self.adj[v_rm].add(u_rm)
        self.deg[u_rm] += 1
        self.deg[v_rm] += 1
        return False, 0.0
    new_S = np.log(1 + new_n_tri) - BETA_VIAB * self._compute_D()
    delta_S = new_S - self.S_curr

    if self.mode == 'null':

```

```

        accepted = self.rng.random() < 0.5
    else:
        log_alpha = self.N * delta_S / self.tau
        accepted = (log_alpha >= 0) or (self.rng.random() < np.exp(max(log_al

if accepted:
    self.edge_list[idx] = (u_add, v_add)
    self.n_tri = new_n_tri
    self.n_tetra += (dtetra_add - dtetra_rm)
    self.S_curr = new_S
    # Comptabilite production entropie
    if delta_S > 0:
        self.q_in += delta_S
    else:
        self.q_out += abs(delta_S)
    if record_delta_S:
        self.delta_S_history.append(delta_S)
    return True, delta_S
else:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False, 0.0

# =====
# T1 - PRODUCTION D ENTROPIE LOCALE (NESS)
# =====
def measure_entropy_production(sim, n_steps, window=10000):
    """
    Mesure sigma_t = (Q_in - Q_out) / temps sur fenetres glissantes.
    Si sigma_t > 0 stable -> systeme hors equilibre stationnaire (NESS).
    Si sigma_t = 0 -> equilibre.
    """
    sigma_history = []
    q_in_prev = sim.q_in
    q_out_prev = sim.q_out

    for step in range(n_steps):
        sim.step_swap()
        if (step + 1) % window == 0:
            dq_in = sim.q_in - q_in_prev

```

```

        dq_out = sim.q_out - q_out_prev
        sigma = (dq_in - dq_out) / window
        sigma_history.append(sigma)
        q_in_prev = sim.q_in
        q_out_prev = sim.q_out

    return sigma_history

# =====
# T2 - DISTRIBUTION P(Delta S) - TEST CROOKS
# =====
def measure_delta_S_distribution(sim, n_steps):
    """Enregistre tous les Delta S des pas acceptes pendant n_steps pas."""
    sim.delta_S_history = []
    for _ in range(n_steps):
        sim.step_swap(record_delta_S=True)
    return list(sim.delta_S_history)

def test_crooks_relation(delta_S_list, n_bins=30):
    """
    Test de la relation de Crooks :  $P(\Delta S) / P(-\Delta S) \sim \exp(N \Delta S / \tau)$ 
    Equivalent a verifier que  $\log[P(+)/P(-)]$  est lineaire en Delta S avec pente
    Si la pente coincide -> equilibre detaille.
    Si elle differe -> hors equilibre.
    """
    delta_S_arr = np.array(delta_S_list)
    if len(delta_S_arr) == 0:
        return None

    # Histogramme symetrique
    abs_max = np.percentile(np.abs(delta_S_arr), 95)
    if abs_max == 0:
        return None

    bins = np.linspace(-abs_max, abs_max, n_bins + 1)
    centers = (bins[:-1] + bins[1:]) / 2

    counts, _ = np.histogram(delta_S_arr, bins=bins)

    # Pour chaque bin positif, comparer avec le bin negatif symetrique
    n_pos = n_bins // 2
    log_ratios = []
    centers_pos = []
    for i in range(n_pos):
        bin_pos = n_bins - 1 - i # bin a Delta S > 0
        bin_neg = i # bin a Delta S < 0 symetrique

```

```

        if counts[bin_pos] > 5 and counts[bin_neg] > 5:
            log_ratios.append(np.log(counts[bin_pos] / counts[bin_neg]))
            centers_pos.append(centers[bin_pos])

    if len(log_ratios) < 3:
        return None

    centers_pos = np.array(centers_pos)
    log_ratios = np.array(log_ratios)

    # Fit lineaire
    slope, intercept = np.polyfit(centers_pos, log_ratios, 1)
    pred = slope * centers_pos + intercept
    ss_res = np.sum((log_ratios - pred)**2)
    ss_tot = np.sum((log_ratios - log_ratios.mean())**2)
    r2 = 1 - ss_res / ss_tot if ss_tot > 0 else 0

    return {
        'slope_measured': float(slope),
        'intercept': float(intercept),
        'r2': float(r2),
        'n_points': len(log_ratios),
    }

# =====
# T3 - REPONSE LINEAIRE
# =====
def measure_linear_response(N, tau, seed, n_equil, n_measure, mode, eps=0.05):
    """
    Mesure la reponse a une perturbation tau -> tau + eps.
    Compare <D> a tau et a tau+eps. La pente <D>(tau+eps) - <D>(tau) / eps = chi_
    """
    # Run a tau (reference)
    sim_ref = BlobUniverseFixedN(N, tau, seed, mode=mode)
    for _ in range(n_equil):
        sim_ref.step_swap()
    D_ref_samples = []
    for _ in range(n_measure):
        sim_ref.step_swap()
        D_ref_samples.append(sim_ref._compute_D())
    D_ref = np.mean(D_ref_samples)

    # Run a tau+eps (perturbe)
    sim_per = BlobUniverseFixedN(N, tau + eps, seed + 100, mode=mode)
    for _ in range(n_equil):
        sim_per.step_swap()

```

```

D_per_samples = []
for _ in range(n_measure):
    sim_per.step_swap()
    D_per_samples.append(sim_per._compute_D())
D_per = np.mean(D_per_samples)

# Coefficient de reponse
chi_DD = (D_per - D_ref) / eps

return {
    'tau_ref': tau,
    'tau_per': tau + eps,
    'D_ref_mean': float(D_ref),
    'D_per_mean': float(D_per),
    'chi_DD': float(chi_DD),
    'eps': eps,
}

# =====
# MAIN
# =====
print('')
print('=' * 75)
print('  BLOB - SAUT 3.7 - THERMODYNAMIQUE NON-EQUILIBRE')
print('=' * 75)

N = int(os.environ.get('BLOB_N', 200))
TAU = float(os.environ.get('BLOB_TAU', 0.50))
N_SEEDS = int(os.environ.get('N_SEEDS', 3))
N_EQUIL = int(os.environ.get('N_EQUIL', 200000))
N_MEASURE = int(os.environ.get('N_MEASURE', 500000))
WINDOW = int(os.environ.get('WINDOW', 10000))
RUN_NULL = os.environ.get('RUN_NULL', '1') == '1'
RUN_T3 = os.environ.get('RUN_T3', '1') == '1'

print(f'  N : {N}')
print(f'  tau : {TAU}')
print(f'  seeds : {N_SEEDS}')
print(f'  equilibration : {N_EQUIL}')
print(f'  mesure : {N_MEASURE}')
print(f'  window sigma : {WINDOW}')
print(f'  Run NULL : {RUN_NULL}')
print(f'  Run T3 (reponse lineaire) : {RUN_T3}')
total_runs = N_SEEDS * (2 if RUN_NULL else 1) + (N_SEEDS * (2 if RUN_NULL else 1))
print(f'  Estimation : ~{total_runs * (N_EQUIL + N_MEASURE) * 1e-5:.0f} min')
print('=' * 75)

```

```

t_global = time.time()

# =====
# T1 + T2 : Production entropie + Crooks (BLOB)
# =====

print('')
print('  === T1 + T2 : MODE BLOB ===')
results_blob = []
for seed in range(N_SEEDS):
    print(f'')
    print(f'  --- Seed {seed} BLOB ---')
    t0 = time.time()

    sim = BlobUniverseFixedN(N, TAU, seed, mode='blob')

    # Equilibration
    print(f'    Equilibration {N_EQUIL} pas...', flush=True)
    for _ in range(N_EQUIL):
        sim.step_swap()

    # Reset compteurs apres equilibration
    sim.q_in = 0.0
    sim.q_out = 0.0
    sim.delta_S_history = []

    # T1 - production entropie + T2 - delta S history simultanement
    print(f'    Mesure {N_MEASURE} pas avec tracking delta_S...', flush=True)
    sigma_history = []
    q_in_prev = 0.0
    q_out_prev = 0.0

    for step in range(N_MEASURE):
        sim.step_swap(record_delta_S=True)
        if (step + 1) % WINDOW == 0:
            dq_in = sim.q_in - q_in_prev
            dq_out = sim.q_out - q_out_prev
            sigma = (dq_in - dq_out) / WINDOW
            sigma_history.append(sigma)
            q_in_prev = sim.q_in
            q_out_prev = sim.q_out

    # T2 - test Crooks
    crooks = test_crooks_relation(sim.delta_S_history)

    elapsed = time.time() - t0
    print(f'    sigma_t (final) = {np.mean(sigma_history):.6e}, sigma_q90 = {np.p

```

```

print(f'      n delta_S enregistres : {len(sim.delta_S_history)}', flush=True)
if crooks:
    slope_expected = N / TAU
    ratio = crooks['slope_measured'] / slope_expected
    print(f'      Crooks: slope = {crooks["slope_measured"]:.2f}, expected (N/t
print(f'      duree {elapsed:.0f}s', flush=True)

results_blob.append({
    'seed': seed, 'mode': 'blob',
    'sigma_history': sigma_history,
    'sigma_mean': float(np.mean(sigma_history)),
    'sigma_std': float(np.std(sigma_history)),
    'q_in_total': sim.q_in, 'q_out_total': sim.q_out,
    'n_delta_S': len(sim.delta_S_history),
    'delta_S_stats': {
        'mean': float(np.mean(sim.delta_S_history)) if sim.delta_S_history el
        'std': float(np.std(sim.delta_S_history)) if sim.delta_S_history else
        'min': float(np.min(sim.delta_S_history)) if sim.delta_S_history else
        'max': float(np.max(sim.delta_S_history)) if sim.delta_S_history else
    },
    'crooks': crooks,
    'elapsed_s': elapsed,
})
gc.collect()

# =====
# T1 + T2 : Production entropie + Crooks (NULL)
# =====
results_null = []
if RUN_NULL:
    print('')
    print(' === T1 + T2 : MODE NULL ===')
    for seed in range(N_SEEDS):
        print(f'')
        print(f' --- Seed {seed} NULL ---')
        t0 = time.time()

        sim = BlobUniverseFixedN(N, TAU, seed, mode='null')

        for _ in range(N_EQUIL):
            sim.step_swap()

        sim.q_in = 0.0
        sim.q_out = 0.0
        sim.delta_S_history = []

        sigma_history = []

```

```

q_in_prev = 0.0
q_out_prev = 0.0

for step in range(N_MEASURE):
    sim.step_swap(record_delta_S=True)
    if (step + 1) % WINDOW == 0:
        dq_in = sim.q_in - q_in_prev
        dq_out = sim.q_out - q_out_prev
        sigma = (dq_in - dq_out) / WINDOW
        sigma_history.append(sigma)
        q_in_prev = sim.q_in
        q_out_prev = sim.q_out

crooks = test_crooks_relation(sim.delta_S_history)
elapsed = time.time() - t0

print(f'    sigma_t = {np.mean(sigma_history):.6e}', flush=True)
if crooks:
    print(f'    Crooks: slope = {crooks["slope_measured"]:.2f}, R2 = {cro

results_null.append({
    'seed': seed, 'mode': 'null',
    'sigma_history': sigma_history,
    'sigma_mean': float(np.mean(sigma_history)),
    'sigma_std': float(np.std(sigma_history)),
    'q_in_total': sim.q_in, 'q_out_total': sim.q_out,
    'n_delta_S': len(sim.delta_S_history),
    'crooks': crooks,
    'elapsed_s': elapsed,
})
gc.collect()

# =====
# T3 - REPONSE LINEAIRE (optionnel, plus court)
# =====
results_T3 = []
if RUN_T3:
    print('')
    print('    === T3 : REPONSE LINEAIRE ===')
    n_equil_T3 = max(N_EQUIL // 4, 50000)
    n_measure_T3 = max(N_MEASURE // 4, 100000)

    for seed in range(N_SEEDS):
        print(f'')
        print(f'    --- Seed {seed} T3 BLOB ---')
        t0 = time.time()
        r = measure_linear_response(N, TAU, seed, n_equil_T3, n_measure_T3, mode=

```

```

        r['seed'] = seed
        r['mode'] = 'blob'
        elapsed = time.time() - t0
        print(f'    chi_DD = {r["chi_DD"]:.4f} (D_ref={r["D_ref_mean"]:.4f}, D_pe
        results_T3.append(r)
        gc.collect()

if RUN_NULL:
    for seed in range(N_SEEDS):
        print(f'')
        print(f' --- Seed {seed} T3 NULL ---')
        t0 = time.time()
        r = measure_linear_response(N, TAU, seed, n_equil_T3, n_measure_T3, m
        r['seed'] = seed
        r['mode'] = 'null'
        elapsed = time.time() - t0
        print(f'    chi_DD = {r["chi_DD"]:.4f} ({elapsed:.0f}s)', flush=True)
        results_T3.append(r)
        gc.collect()

total_time = time.time() - t_global
print('')
print('=' * 75)
print(f' Toutes mesures terminees en {total_time/60:.1f} min')
print('=' * 75)

# =====
# ANALYSE DES VERDICTS
# =====
print('')
print('=' * 75)
print(' VERDICTS')
print('=' * 75)

# T1 - production entropie
print('')
print(' T1 - PRODUCTION D ENTROPIE LOCALE (sigma_t)')
sigma_blob = [r['sigma_mean'] for r in results_blob]
sigma_blob_mean = np.mean(sigma_blob)
print(f'    sigma_t Blob : moyenne sur seeds = {sigma_blob_mean:.6e}')
print(f'    std = {np.std(sigma_blob):.6e}')
if RUN_NULL:
    sigma_null = [r['sigma_mean'] for r in results_null]
    sigma_null_mean = np.mean(sigma_null)
    print(f'    sigma_t Null : moyenne sur seeds = {sigma_null_mean:.6e}')
    if abs(sigma_null_mean) > 1e-9:
        ratio = sigma_blob_mean / sigma_null_mean

```

```

        print(f'    Ratio Blob/Null = {ratio:.3f}')
print('')
abs_sigma_blob = abs(sigma_blob_mean)
if abs_sigma_blob > 1e-5:
    print('    >>> sigma_t significativement different de zero -> NESS confirme')
elif abs_sigma_blob > 1e-7:
    print('    >>> sigma_t faible mais detectable -> NESS faible / quasi-equilibr
else:
    print('    >>> sigma_t ~ 0 -> systeme proche de l equilibre')

# T2 - Crooks
print('')
print('  T2 - RELATION DE CROOKS')
slope_expected = N / TAU
print(f'    Pente attendue a l equilibre detaille (N/tau) : {slope_expected:.1f}')
crooks_blob_slopes = []
for r in results_blob:
    if r['crooks']:
        crooks_blob_slopes.append(r['crooks']['slope_measured'])
if crooks_blob_slopes:
    cb_mean = np.mean(crooks_blob_slopes)
    cb_std = np.std(crooks_blob_slopes)
    ratio = cb_mean / slope_expected
    print(f'    Pente mesuree Blob : {cb_mean:.2f} +/- {cb_std:.2f}')
    print(f'    Ratio mesure/attendu : {ratio:.3f}')
    if 0.85 <= ratio <= 1.15:
        print('    >>> Crooks respecte (ratio ~ 1) -> equilibre detaille local OK
    else:
        print('    >>> Crooks viole -> hors equilibre')

if RUN_NULL:
    crooks_null_slopes = []
    for r in results_null:
        if r['crooks']:
            crooks_null_slopes.append(r['crooks']['slope_measured'])
    if crooks_null_slopes:
        cn_mean = np.mean(crooks_null_slopes)
        print(f'    Pente mesuree Null : {cn_mean:.2f}')
        print(f'    (rappel : Null = acceptance 50% -> Crooks attendu trivialemen

# T3 - reponse lineaire
if RUN_T3:
    print('')
    print('  T3 - REPONSE LINEAIRE')
    chi_DD_blob = [r['chi_DD'] for r in results_T3 if r['mode'] == 'blob']
    chi_DD_null = [r['chi_DD'] for r in results_T3 if r['mode'] == 'null']
    if chi_DD_blob:

```

```

        print(f'      chi_DD Blob : {np.mean(chi_DD_blob):.4f} +/- {np.std(chi_DD_b
if chi_DD_null:
        print(f'      chi_DD Null : {np.mean(chi_DD_null):.4f} +/- {np.std(chi_DD_n

# Verdict global
print('')
print('=' * 75)
print('  VERDICT GLOBAL')
print('=' * 75)
if abs_sigma_blob > 1e-5:
    if crooks_blob_slopes and 0.85 <= np.mean(crooks_blob_slopes) / slope_expecte
        print('  Le Blob est en regime QUASI-NESS : production entropie > 0 mais
        print('    respecte. Caracterisation : systeme stationnaire avec flux d ent
    else:
        print('  Le Blob est en NESS classique : production entropie > 0 et Crook
        print('    Caracterisation : systeme hors-equilibre pleinement non-detaille
else:
    print('  Le Blob est proche de l equilibre statistique.')
    print('  Le ratio Q3 Blob < Q3 Null pourrait etre du au caractere selectif de

# =====
# SAUVEGARDE
# =====
report = {
    'config': {'N': N, 'tau': TAU, 'n_seeds': N_SEEDS, 'n_equil': N_EQUIL,
               'n_measure': N_MEASURE, 'window': WINDOW, 'run_null': RUN_NULL,
               'run_T3': RUN_T3, 'total_time_s': total_time},
    'results_blob': results_blob,
    'results_null': results_null if RUN_NULL else [],
    'results_T3': results_T3 if RUN_T3 else [],
}

# Drop sigma_history details to limit file size if too big
for r in report['results_blob']:
    if 'sigma_history' in r and len(r['sigma_history']) > 500:
        r['sigma_history_summary'] = {
            'n_points': len(r['sigma_history']),
            'mean': float(np.mean(r['sigma_history'])),
            'std': float(np.std(r['sigma_history'])),
            'first_10': r['sigma_history'][:10],
            'last_10': r['sigma_history'][-10:],
        }
        del r['sigma_history']
for r in report['results_null']:
    if 'sigma_history' in r and len(r['sigma_history']) > 500:
        r['sigma_history_summary'] = {
            'n_points': len(r['sigma_history']),

```

```

        'mean': float(np.mean(r['sigma_history'])),
        'std': float(np.std(r['sigma_history'])),
        'first_10': r['sigma_history'][:10],
        'last_10': r['sigma_history'][-10:],
    }
    del r['sigma_history']

report_path = OUTPUT_DIR + '/saut37_RAPPORT.json'
with open(report_path, 'w') as f:
    json.dump(report, f, indent=2, default=str)
print('')
print(f' Rapport : {report_path}')
print(f' Taille : {os.path.getsize(report_path)} octets')

```